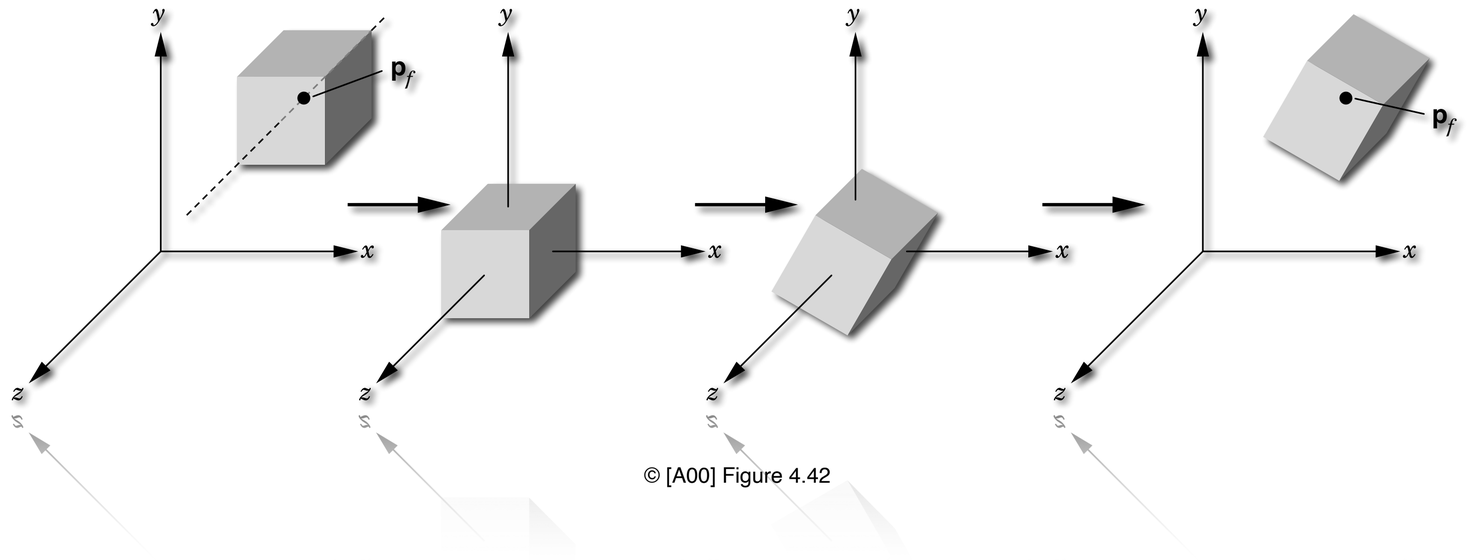
Compositions of Transformations



© [A00] Figure 4.42

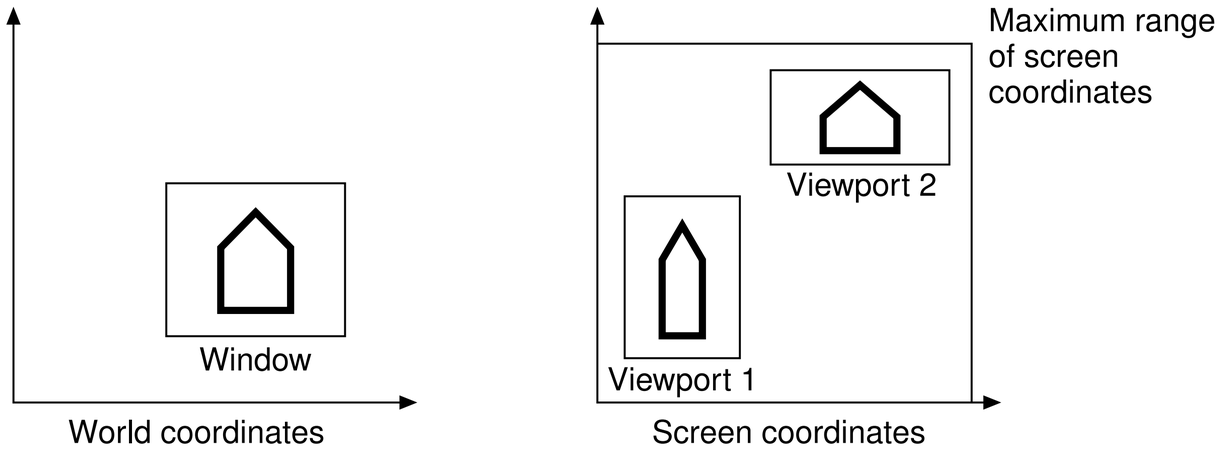
Composition of Transforms

- Multiple fundamental transformations T_i can be composed into one single transformation by matrix multiplication:

$$M = T_\mathrm{k} \cdot T_\mathrm{k\text{-}1} \ldots T_2 \cdot T_1$$

  - note the right-to-left order, $\boldsymbol{T_i}$ is applied before $\boldsymbol{T_{i+1}}$ on the object
  - gain efficiency by combining transformations applied to many objects or elements
  - order of transformations is important as matrix multiplication is **NOT** commutative in general
    - only very few specific transformations (e.g. **uniform** scaling) can partially be reordered

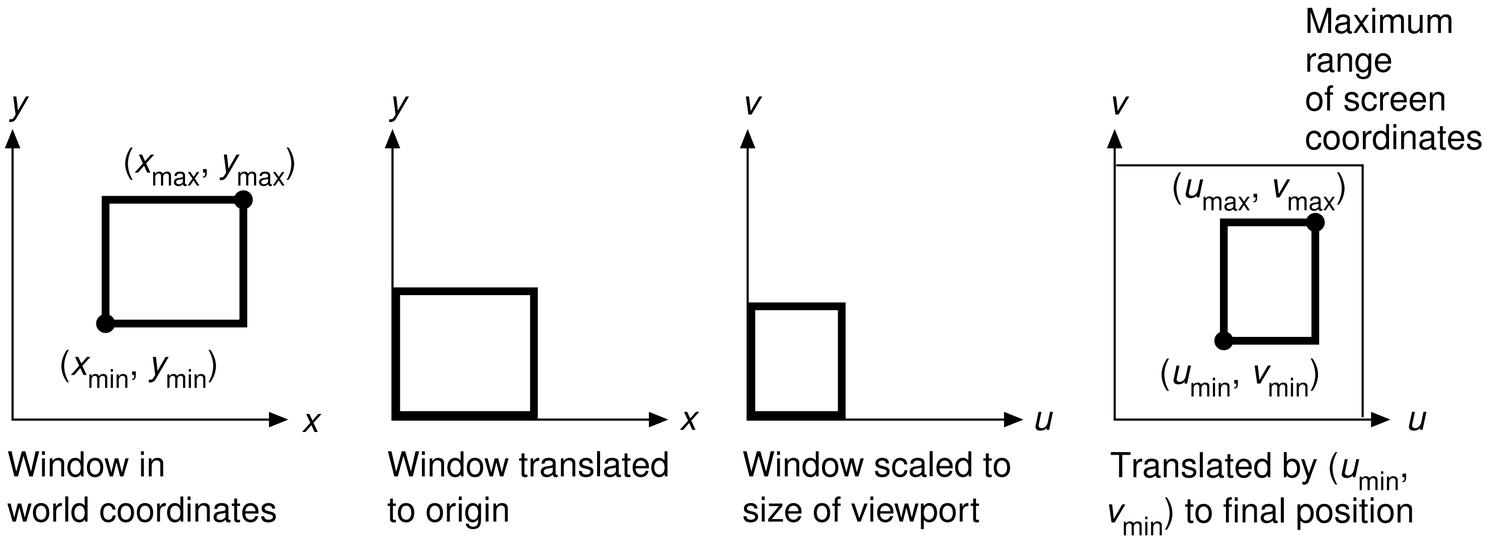
2D Example: Window-to-Viewport



© [AW94] Figures 5.12 and 5.13

- Map the 3D scene's window **coordinates** into device or **screen coordinates**
  - specified by world-coordinate window and corresponding rectangular viewport region on screen
    - applied to all output primitives in world window coordinates
  - viewing window and viewport may have different **aspect ratios**

Window-to-Viewport Mapping



© [AW94] Figure 5.14

- Window-to-viewport mapping matrix M_{WV} is composed of:
  1. Translation of world coordinate window to origin
  2. Scaling to size of viewport rectangle
  3. Translation of viewport to final position on screen

$$M_{wv} = T(u_{min}, v_{min}) \cdot S\left(\frac{u_{max}-u_{min}}{x_{max}-x_{min}}, \frac{v_{max}-v_{min}}{y_{max}-y_{min}}\right) \cdot T(-x_{min}, -y_{min})$$

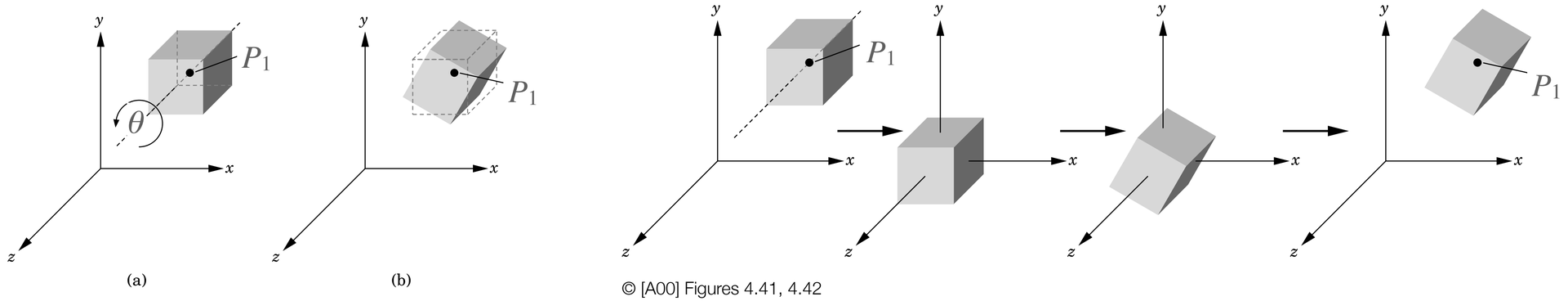
$$= \begin{bmatrix} 1 & 0 & u_{min} \\ 0 & 1 & v_{min} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \frac{u_{max}-u_{min}}{x_{max}-x_{min}} & 0 & 0 \\ 0 & \frac{v_{max}-v_{min}}{y_{max}-y_{min}} & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -x_{min} \\ 0 & 1 & -y_{min} \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \frac{u_{max}-u_{min}}{x_{max}-x_{min}} & 0 & -x_{min} \cdot \frac{u_{max}-u_{min}}{x_{max}-x_{min}} + u_{min} \\ 0 & \frac{v_{max}-v_{min}}{y_{max}-y_{min}} & -y_{min} \cdot \frac{v_{max}-v_{min}}{y_{max}-y_{min}} + v_{min} \\ 0 & 0 & 1 \end{bmatrix}$$

- Transformation of points $P=(x,y,1)$ by M_{WV} yields expected result:

$$P' = \left((x - x_{min}) \cdot \frac{u_{max}-u_{min}}{x_{max}-x_{min}} + u_{min}, (y - y_{min}) \cdot \frac{v_{max}-v_{min}}{y_{max}-y_{min}} + v_{min}, 1\right)$$

Composition Example 1

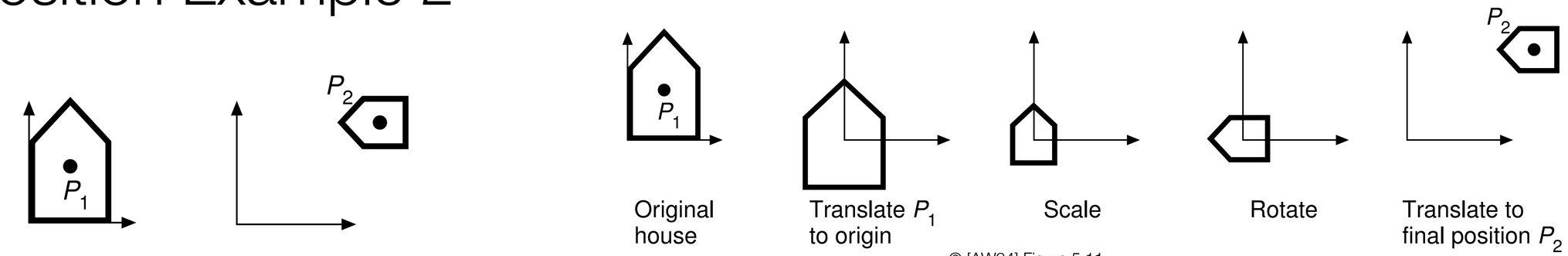


© [A00] Figures 4.41, 4.42

- Rotation about arbitrary P_1 can be composed from a few consecutive fundamental transformations
  1. Translation of P_1 to the origin
  2. Rotation e.g. about z-axis (at the origin)
  3. Translate P_1 back to its initial location
- *2D Example*: Rotation origin is (x_1, y_1) and rotation angle θ: $T(x_1, y_1)\cdot R(\theta)\cdot T(-x_1,-y_1)$

$$M = \begin{bmatrix} 1 & 0 & x_1 \\ 0 & 1 & y_1 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -x_1 \\ 0 & 1 & -y_1 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & x_1(1-\cos\theta)+y_1\sin\theta \\ \sin\theta & \cos\theta & y_1(1-\cos\theta)-x_1\sin\theta \\ 0 & 0 & 1 \end{bmatrix}$$

Composition Example 2



© [AW94] Figure 5.11

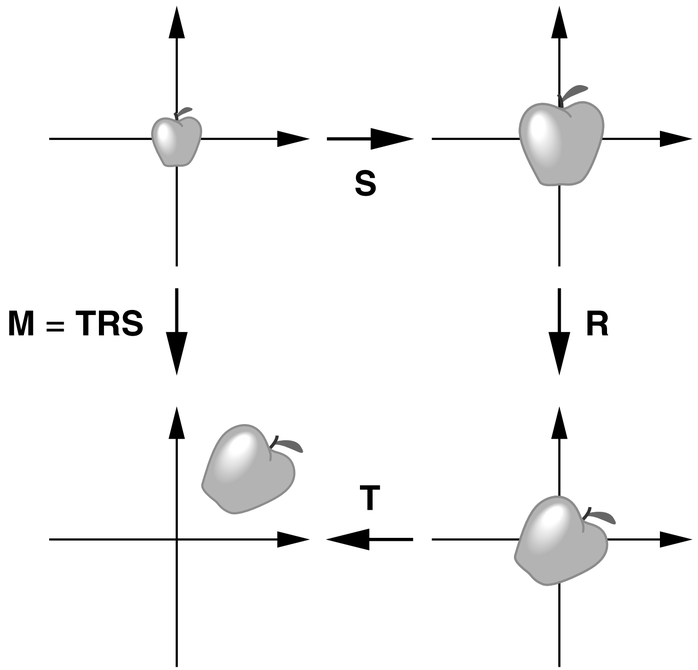
- Scaling **and** rotation about arbitrary anchor point
  - anchor point should not undergo any side-effect translation
  1. Translation of P_1 to the origin
  2. Scaling by s_x and s_y
  3. Rotation 90° counter clock-wise
  4. Translate back to (new) location P_2
- *2D Example*: Scaling origin is center point (x_1, y_1): $T(x_2, y_2) \cdot R(90°) \cdot S(s_x,s_y) \cdot T(-x_1,-y_1)$

$$M = \begin{bmatrix} 1 & 0 & x_2 \\ 0 & 1 & y_2 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -x_1 \\ 0 & 1 & -y_1 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -s_x & x_1 + s_x y_1 \\ s_y & 0 & y_1 - s_y x_1 \\ 0 & 0 & 1 \end{bmatrix}$$

SRT Standard

- Typical combination of transformations includes selecting the scale of the object, its orientation in space and position of the object at the desired location

- SRT: **scale – rotate – translate**
  - in matrix order (right to left!): $\boldsymbol{T \bullet R \bullet S}$
  - but possibly first set scale/rotation anchor point (x_1, y_1)



© [A00] Figure 4.47

Implementation

- Any translation, scaling, and rotation transformations can be combined together in a **composite matrix**
  - **upper-left 3x3** submatrix R is

    aggregate rotation and scaling
  - **upper-right 3x1** column vector T

    represents **aggregate translation**

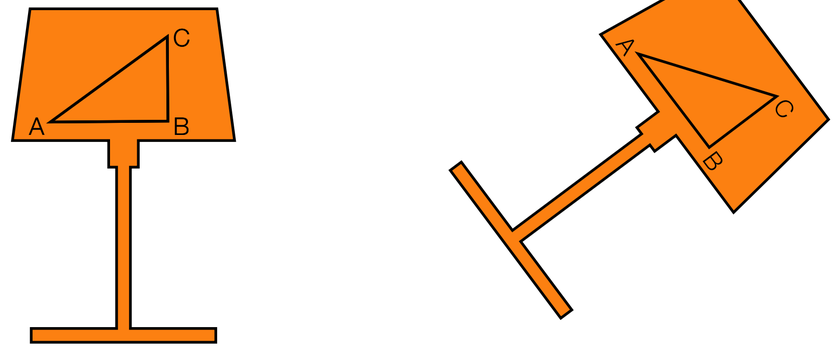
$$M = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = T \cdot R \cdot S$$

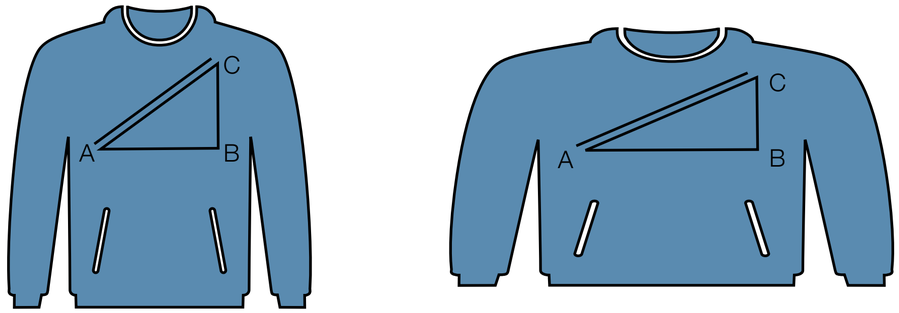
- Apply rotation/scale and translation separately for efficiency
  - 9 multiplications and 9 additions (instead of 16 and 12)

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = R \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} + T.$$

Transformation Properties

- **Rigid-body** transformations
  - preserving lengths and angles, no object distortion
- **Affine** transformations
  - preserving parallelism
  - non-uniform scaling is affine but not a rigid-body transform
- Scaling and rotation are dependent on the origin
  - translation is not
  - scaling (and rotation) is an operation on vectors
    - or on position vectors
  - translation is an operation on points





Inverse

- The transformation, scaling, and rotation matrices all have inverses
  - not true for all general matrices
- The inverses are formed by:

  negation of displacements $d_x, d_y, d_z \rightarrow -d_x, -d_y, -d_z$

  reciprocal of scale values $s_x, s_y, s_z \rightarrow 1/s_x, 1/s_y, 1/s_z$

  negation of rotation angle $\theta \rightarrow -\theta$

- Inversion **of any orthogonal matrix** B is equivalent to **transposition** $B^{-1}=B^T$
  - works well for the special orthogonal rotation matrices
  - consistent with negation of rotation angles

Inverse Composition

- Inversion of a sequence of transformations is done by:
  inverting the **sequence order** and
  inverting each **transformation** individually

$$M = M_4 \cdot M_3 \cdot M_2 \cdot M_1$$

$$M^{-1} = M_1^{-1} \cdot M_2^{-1} \cdot M_3^{-1} \cdot M_4^{-1}$$

Exercise:Transformation of Object Normals

- Geometric objects defined by points are transformed by transforming their points directly:

$$P' = M \cdot P$$

- **Transformation of a plane normal (or plane coefficients) is different**
  - plane **equation** $\boldsymbol{Ax + By + Cz + D = 0}$ defined by generalized plane parameters $N = (A, B, C, D)$ and for any point $P = (x, y, z, 1)$ on the plane
  - for any point $P = (x, y, z, 1)$ on the plane it is **equivalent to** $\boldsymbol{N^{\mathrm{T}} \cdot P = 0}$, using the dot-product of N and P

- Normal transformed by Q as: $N' = Q \cdot N$
  - transformation Q of normals is not the same as M
  - it must maintain that $\boldsymbol{N'^{\mathrm{T}} \cdot P' = 0}$ for all transformed points $P' = M \cdot P$

Exercise:Transforming Object Normals

- The plane equation

$$N'^{\mathrm{T}} \cdot P' = (Q \cdot N)^{\mathrm{T}} \cdot M \cdot P = N^{\mathrm{T}} \cdot Q^{\mathrm{T}} \cdot M \cdot P = 0$$

  holds iff $Q^{\mathrm{T}} \cdot M$ **is a multiple of the identity matrix**

  - Using $(Q \cdot N)^{\mathrm{T}} = N^{\mathrm{T}} \cdot Q^{\mathrm{T}}$, and since $N^{\mathrm{T}} \cdot P = 0$

- Thus from $Q^{\mathrm{T}} \cdot M = I$ we get that $Q = (M^{-1})^{\mathrm{T}}$
  - note that M^{-1} may not exist in general due to zero determinant (e.g. if including a projection)
  - however, if M consists only of fundamental transforms (such as e.g. translate, scale, rotate, shear, reflect) then M^{-1} exists and can be formed knowing the composition of M
- **Transform points by M, and transform normals by Q**
  - transform vectors correctly by transforming its endpoints

Transformation Task

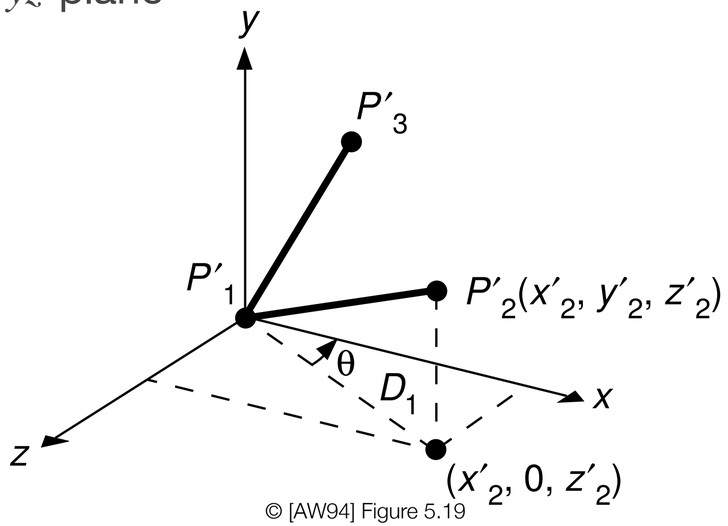
(a) Initial position

(b) Final position

- Transform P_1 to origin, P_2 onto z-axis and P_3 into yz-plane using rigid body transforms
- **First approach**: compose from four fundamental transforms
  1. translate P_1 to origin
  2. rotate around the y-axis to move P_2 into the yz-plane
  3. rotate around the x-axis to align P_2 with the z-axis
  4. rotate around the z-axis to move P_3 into the yz-plane

Transformation Task 1a

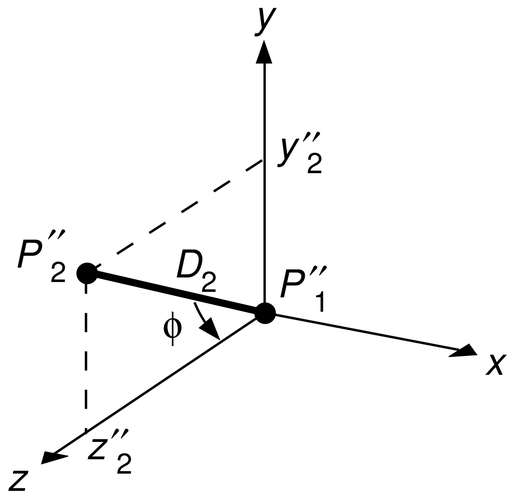
- Translation (1): $T(-x_1, -y_1, -z_1)$ is simply given by the negative coordinates of P_1
- Rotation (2): $R_y(\theta\text{-}90)$ is defined by the angle θ between the projection D_1 of P_1'P_2' in the xz-plane and the x-axis
  - rotate about y-axis to map P_2 into the yz-plane



© [AW94] Figure 5.19

Transformation Task 1a

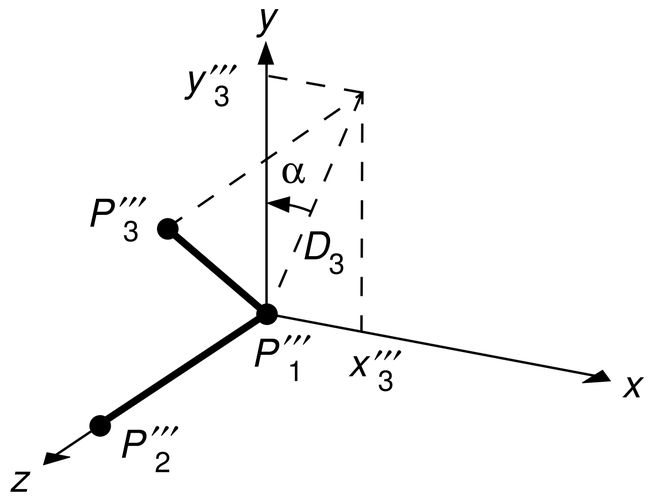
- Rotation (3): $R_x(\phi)$ is defined by the angle ϕ of $D_2 = P_1'' P_2''$ in the the yz-plane and the z-axis
  - rotate about x-axis to align P_2 with z-axis



© [AW94] Figure 5.20

Transformation Task 1a

- Rotation (4): $R_z(\alpha)$ is defined by the angle α between the projection D_3 of $P_1'''P_3'''$ in the xy-plane and the y-axis
  - rotate about z-axis to map P_3 into yz-plane



© [AW94] Figure 5.21

- **Final composite:** $M = R_z(\alpha) \cdot R_x(\phi) \cdot R_y(\theta - 90) \cdot T(-x_1,-y_1,-z_1)$

Copyrights

- Many figures in these slides are copyright protected as indicated. Text and all other figures are copyright protected by Renato Pajarola.

- [AS12] Electronic Book Support for Interactive Computer Graphics: A Top-Down Approach with Shader-based OpenGL by Edward Angel and Dave Shreiner. Copyright Addison Wesley 2012.

- [A00] Electronic Book Support for Interactive Computer Graphics: A Top-Down Approach Using OpenGL by Edward Angel. Copyright E. Angel 2000. (http://www.cs.unm.edu/~angel/BOOK/SECOND_EDITION/)

- [AW94] Electronic Instructors Manual (EIM) for Introduction to Computer Graphics by James Foley, Andries van Dam, Steven Feiner, John Hughes, and Richard Phillips. Copyright Addison Wesley 1994.

- [AC97] Learning Rendering CD accompanying 3D Computer Graphics by Alan Watt. Copyright A. Watt and L. Cooper 1997. (Permission for copying and distribution without direct commercial advantage. To copy otherwise, or to republish, requires specific permission.)

- [S02] Electronic Figures for Fundamentals of Computer Graphics by Peter Shirley. Copyright P. Shirley 2002. (http://www.cs.utah.edu/~shirley/fcg/figures/)